

act like a senior Front-end Engineer. I want to make my portfolio. I want this section Home, About, Skills, Experience, Projects, Blog, Contract and Faq. I'm Towhid Ahmed. I'm an AI/ML Engineer | Generative AI Engineer. I give you 1 picture and 2 pdf cv file. First is for home page and second is ai AI/ML Engineer Resume and third is Generative AI Engineer Resume.

Home section write AI/ML Engineer | Machine Learning | NLP & Generative AI | RAG Pipeline | LLM | Python | Strong in DSA (Java) | Open to Remote Opportunities. Add 2 beautiful button for download my AI/ML Engineer CV and other button for Generative AI Engineer CV.

About section you have to write: I am Towhid Ahmed, an AI/ML & Generative AI Engineer who independently builds and deploys production-grade intelligent systems for high-velocity, remote-first teams. I specialize in the end-to-end ML lifecycle, with core expertise in NLP, multi-agent workflows, and advanced RAG pipelines using Python, FastAPI, and Docker. Driven by a production-first mindset, I focus on mitigating hallucinations and optimizing low-latency retrieval performance, all backed by a strong problem-solving foundation of 400+ DSA problems solved in Java.

Skills section you have to write :

Front-end: Next.js, React, HTML, CSS & JavaScript, Streamlit.

Backend: Flask, FastApi,

Dadabase & Authentication: JWT / OAuth2 , PostgreSQL / SQL

Programming Languages & Data: Python (Expert), Java (DSA), PostgreSQL.

Machine Learning & Deep Learning: Supervised & Unsupervised Learning, EDA, Data Preprocessing, Feature Engineering, **Regression & Classification** Algorithms, KNN, Decision Trees, Random Forest, Naive Bayes, **TensorFlow, Keras, Scikit-learn, CNN, OpenCV** RNN/LSTM, Model Evaluation.

NLP & Generative AI: LLMs, **Prompt Engineering, RAG Pipelines, Vector Databases** (FAISS, ChromaDB, Weaviate), Sentence Transformers, **Cross-Encoder Re-ranking, LangChain**, LLaMA, HuggingFace, BERT, Transformers, **Multi-Agent Systems, AI Agents**.

MLOps & Deployment: MLflow, DVC, DagsHub, **Docker, FastAPI**, Streamlit, **REST APIs, CI/CD, Render**, Railway.

Software Engineering: 400+ DSA problems solved, Object-Oriented Programming (OOP), System Design

Developer Tools: Pandas, NumPy, Next.js, GitHub, Claude, GitHub Copilot, Cursor AI, Groq API, Hugging Face Hub

Experience: In this section you write experience project based.

AI/ML Engineer (Project-Based Experience)

Self-Driven | Production-Grade AI Systems

Built and deployed multiple end-to-end machine learning and generative AI systems with a strong focus on production readiness, scalability, and real-world deployment.

Designed and implemented full ML pipelines covering data preprocessing, feature engineering, model training, evaluation, and deployment using Python, Scikit-learn, TensorFlow, FastAPI, and Docker.

Developed production-level NLP and Generative AI applications including RAG pipelines, multi-agent systems, and LLM-powered assistants with tools like LangChain, ChromaDB, FAISS, and Weaviate.

Engineered and deployed systems such as spam detection, recommendation engines, medical assistants, AI IDEs, and research assistants with monitoring, CI/CD, and cloud deployment.

Focused on building reliable, low-latency, and scalable AI systems while applying strong software engineering principles and solving 400+ DSA problems in Java.

Projects:

I give here my project details with github and demo link. You have to amazing this section with your best. In project you have to show only projects title. Then Add a something like button in every project. When i click the button my project details open.

End-to-End MLOps Email Spam Detection System — scikit-learn | MLflow | DagsHub | FastAPI | PostgreSQL | Alembic | Prometheus | Docker | GitHub Actions (CI/CD) | Pytest **Demo | GitHub**

Built a production-grade email spam detection system with automated retraining, real-time monitoring, and CI/CD deployment on Render.

- Developed a TF-IDF + Logistic Regression classifier achieving 98.5% accuracy and 96% F1-score on the SMS Spam Collection dataset.
- Implemented full MLOps pipeline with MLflow and DagsHub for experiment tracking, model registry, and versioning.
- Designed a feedback-driven retraining system where user corrections stored in PostgreSQL trigger automated model retraining.
- Built RESTful APIs with FastAPI supporting single and batch predictions (up to 50 inputs) with **<5ms latency**; integrated Prometheus for metrics monitoring and Sentry for error tracking.
- Containerized with Docker and deployed via GitHub Actions CI/CD pipeline (lint, test, build, deploy) on Render.

<https://sms-spam-classifier-mlops.onrender.com>

https://github.com/TowhidAhmedd/SMS_Spam_Classifier_MLOPS

Boston House Price Prediction using Machine Learning

- **This project focuses on predicting housing prices using the Boston Housing dataset. It applies supervised machine learning techniques to learn patterns from historical data and generate accurate price predictions.**
- **The project follows a structured ML workflow including data preprocessing, feature engineering, model training, evaluation, and deployment.**
- **It is built using modular code structure and demonstrates practical skills in building and deploying ML models.**

<https://boston-house-price-y76a.onrender.com>

<https://github.com/TowhidAhmedd/Boston-House-Price>

End-to-End Machine Learning Pipeline with MLflow & Render CI/CD

- This project demonstrates a complete end-to-end Machine Learning workflow, starting from data ingestion to model deployment in a production environment.
- It includes structured ML pipelines, experiment tracking using MLflow, and automated CI/CD deployment using render services and GitHub Actions.
- The system is designed with modular architecture for scalability and maintainability, following industry-level MLOps practices.

<https://github.com/TowhidAhmedd/Production-Ready-MLOps-Pipeline-with-ETL>

Potato Disease Classification — Python | TensorFlow | Keras | CNN | OpenCV | NumPy | Streamlit **Demo | GitHub**

Built a CNN classifier with TensorFlow & Keras achieving **97.8% test accuracy**, trained with augmentation, conv/pooling layers, Adam optimizer, and cross-entropy loss.

- Built an optimized data pipeline with resizing, batching, and `cache()/prefetch()` functions; sourced and preprocessed the Kaggle PlantDisease dataset into structured batches.
- Deployed a live Streamlit app on Render for real-time potato leaf disease detection.

<https://potato-disease-classification-using-deep-m67m.onrender.com>

<https://github.com/TowhidAhmedd/Potato-Disease-Classification-using-Deep-Learning>

Movie Recommendation System (ML+NLP) — Python, Scikit-learn, Pandas, NumPy, TF-IDF, Cosine Similarity, Streamlit, TMDB API

Demo | GitHub

Built a content-based movie recommendation system using NLP to analyze movie metadata and deliver personalized top-10 results.

- Applied TF-IDF vectorization and cosine similarity to measure content similarity between movies for accurate suggestions.
- Integrated the TMDB API to fetch real-time movie titles, posters, and metadata for a dynamic experience.
- Deployed the application on Render with public cloud access and an interactive sidebar interface.

<https://movie-recommendation-system-cc8s.onrender.com>

<https://github.com/TowhidAhmedd/Movie-Recommendation-System>

AI Video Assistant — Meeting Intelligence | Python · Streamlit · Whisper · Mistral · LangChain · ChromaDB · Render

- Architected an end-to-end meeting intelligence pipeline — audio ingestion (yt-dlp + FFmpeg), chunked Whisper transcription, and Mistral-powered extraction of summaries, action items, and key decisions via LangChain
- Engineered a RAG Q&A engine over meeting transcripts using ChromaDB and SentenceTransformer embeddings, enabling conversational grounding on full session content
- Deployed on Render via Docker to resolve FFmpeg dependency constraints, with persistent disk-backed Chroma storage and Streamlit serving the full UI from a single Web Service

<https://ai-video-assistant-with-rag-1.onrender.com>

<https://github.com/TowhidAhmedd/AI-Video-Assistant-With-RAG>

Advanced RAG-Based ML Research Assistant — Weaviate | Sentence Transformers | OpenAI API | Groq | Streamlit | Render

Demo | GitHub

Built an end-to-end RAG pipeline for semantic search and Q&A over arXiv ML research papers using Weaviate vector database and OpenAI/Groq LLMs.

- Implemented multiple document chunking strategies (fixed-size, sentence-based, semantic) with sentence-transformer embeddings for high-quality vector indexing.
- Improved retrieval accuracy using cross-encoder re-ranking and evaluated system performance with 5 key metrics — Precision, Recall, MRR, NDCG, and F1 Score.
- Deployed the Streamlit application on Render, enabling public cloud access with a production-ready deployment setup.

<https://advanced-rag-ml-research-assistant-1.onrender.com>

https://github.com/TowhidAhmedd/Advanced_Rag_ML_Research_Assistant

Multi-Agent AI Research System — Python | LangChain | Groq | Tavily API | BeautifulSoup | Streamlit

Demo | GitHub

Built a multi-agent AI pipeline that autonomously searches the web, scrapes content, generates structured research reports, and self-critiques output quality using LangChain and Groq LLM.

- Designed modular agents — Search Agent (Tavily API), Reader Agent (BeautifulSoup scraping), Writer Chain (structured report generation), and Critic Chain (quality scoring and feedback).

- Developed an end-to-end CLI pipeline and an interactive Streamlit UI for exploring the full research workflow on any user-defined topic.

<https://multi-agent-ai-research-system-1.onrender.com>

<https://github.com/TowhidAhmedd/Multi-Agent-AI-Research-System->

Medical Assistant — LLM-Powered Chatbot Python · FastAPI · LangChain · ChromaDB · Groq · HuggingFace · Tavily API

Architected a dual-retrieval RAG pipeline combining local ChromaDB (MMR) and Tavily web search scoped to authoritative medical sources (NIH, WHO, Mayo Clinic), with automated emergency symptom escalation

- Engineered a stateful LangChain inference pipeline with sliding-window conversation memory (k=8), HuggingFace embeddings, and per-response source attribution grounded in retrieved context
- Exposed a multi-modal document ingestion system via REST API, UI upload, and folder watch — chunking and indexing PDFs/TXTs into a persistent vector store at runtime

<https://medical-ai-chatbot-xjto.onrender.com>

https://github.com/TowhidAhmedd/Medical_AI_Chatbot

AI IDE — Multi-Agent Coding Assistant | Python · FastAPI · Groq · FAISS · React · Render

Built a Cursor-like AI coding assistant with a multi-agent backend (Planner, Coder, Debugger, Researcher) using Groq's llama-3.3-70b, real-time SSE token streaming, and a Monaco Editor frontend — deployed end-to-end on Render

- Engineered a RAG pipeline (FAISS + SentenceTransformers) for context-aware codebase memory and a 11-endpoint REST API covering the full coding workflow — generation, debugging, refactoring, and planning
- Integrated Tavily web search into the Research Agent for real-time, AI-synthesized answers grounded in live results

<https://ai-ide-frontend2.onrender.com>

https://github.com/TowhidAhmedd/AI_IDE

Blog:

Generative AI Explained: How Modern AI Creates Text, Images, and Code

Deep dive into LLMs, Transformers, Diffusion Models, and real-world AI systems.

Read Blog:

<https://medium.com/@towhidahmed422/generative-ai-explained-how-modern-ai-creates-text-images-and-code-complete-guide-for-d663e893e600>

Tags: Generative AI | LLM | Machine Learning

How I Approach Building Real-World AI Systems as an ML Engineer

Read Blog:

<https://medium.com/@towhidahmed422/how-i-approach-building-real-world-ai-systems-as-an-ml-engineer-e99abfc9b452>

Tags: Machine Learning AI Engineering MLOps System Design FastAPI

Building an End-to-End Machine Learning Project — From Data to Deployment

Read Blog:

<https://medium.com/@towhid4635/building-an-end-to-end-machine-learning-project-from-data-to-deployment-5213707a4d64>

Tags: Machine Learning FastAPI MLOps Python

Understanding RAG (Retrieval-Augmented Generation) — The Future of LLM Applications

Read Blog:

<https://medium.com/@towhid4635/understanding-rag-retrieval-augmented-generation-the-future-of-llm-applications-85ebad32d0c8>

Tags: RAG (Retrieval Augmented Generation) Large Language Models (LLMs) Vector Databases Embeddings

Contract: my contact info: <https://www.linkedin.com/in/towhid-ahmed-5ba096354/>

<https://github.com/TowhidAhmedd>

<https://x.com/towhid744>

mdt169068@gmail.com

+880 1871 834-635

Email, message, submit button

FAQ:

1. Question: Tell me about yourself

Answer: I am an aspiring AI/ML Engineer with a strong focus on building end-to-end machine learning and NLP-based systems. I have worked on projects like spam detection and recommendation systems, where I handled data preprocessing, model training, and evaluation. I also have strong problem-solving skills, solving 400+ DSA problems. Currently, I am exploring Generative AI and working on building real-world AI applications.

2. Question: Are you working in any timezone?

Answer: I am based in Bangladesh (GMT+6), but I am flexible and comfortable working in different time zones depending on team requirements.

3. Question: Have you worked with any team?

Answer: I have primarily worked independently on my projects, which helped me develop strong self-learning and problem-solving skills. However, I am comfortable collaborating with teams and using tools like GitHub for version control and project management.

4. Question: Are you open to remote work?

Answer: Yes, I am open to remote opportunities and comfortable working in distributed teams.

5. Question: What tools do you use for collaboration?

Answer: I use Git and GitHub for version control and collaboration. I am also familiar with communication tools like Slack, Trello, and Jira.

6. Question: How do you manage your time?

Answer: I prioritize tasks based on importance and deadlines. I break down work into smaller tasks and follow a structured approach to complete projects efficiently.

7. Question: Are you comfortable learning new technologies?

Answer: Yes, I actively learn new technologies, especially in AI/ML and Generative AI.

8. Question: Why should we hire you?

Answer: I have hands-on experience building end-to-end AI/ML projects and strong problem-solving skills. I am highly motivated and focused on building real-world impactful AI solutions.

Create a floating contact button section for a portfolio website.

Requirements:

- Add two circular floating buttons fixed at the bottom of the screen.

- Rightside button: WhatsApp logo with link to my WhatsApp number.

- Right side button: LinkedIn logo with link to my LinkedIn profile.
- Both buttons should always stay visible (position: fixed).
- Add hover effect (slight zoom or scale).
- Buttons should have shadow and modern UI design.
- Make it responsive for mobile and desktop.

WhatsApp link format: <https://wa.me/your-number>

LinkedIn link: <https://linkedin.com/in/your-profile>

Create an interactive FAQ (Frequently Asked Questions) section for a personal portfolio website.

Requirements:

- Display all questions first in a clean list or cards.
- Each question should be clickable.
- When a question is clicked, show/hide the answer (toggle accordion effect).
- Only the answer of the clicked question should expand.
- Smooth animation when expanding/collapsing.
- Modern and minimal UI design.
- Mobile responsive design.

Add GenAI Engineer after AI/ML Engineer home page like AI/ML Engineer | GenAI Engineer. Add hover in my picture social media link, project and every where. when i scroll down the upper section will be go and when scroll up even little upper section will come.

Create a professional AI/ML portfolio website for Towhid Ahmed (Bangladesh), including all sections. Fix grammar, make it recruiter-ready, and design a modern responsive UI. Add any design to make more attractive and professional. If any conflict you choose the best decisions.